

Article

Pattern-Based Automation of User Stories and Gherkin Scenarios from BPMN Models for Agile Requirement [†]

Daniel Mateus ¹, Denis Silva da Silveira ^{2,*}  and João Araujo ¹

¹ Department of Informatics, Faculty of Science and Technology, Universidade Nova de Lisboa, 2829-516 Caparica, Portugal; joao.araujo@fct.unl.pt (J.A.)

² Department of Administration, Graduate Program in Administration, Universidade Federal de Pernambuco, Recife 50670-901, PE, Brazil

* Correspondence: dsilveira@ufpe.br

[†] The introduction of this article explicitly references the short paper entitled “A Systematic Approach to Derive User Stories and Gherkin Scenarios from BPMN Models”, which was published and presented at the International Symposium on Business Modeling and Software Design (BMSD), Utrecht, The Netherlands, 3–5 July 2023.

Abstract

This study enhances agile development by integrating BPMN modeling with automated functional requirements elicitation. It focuses on extracting user stories and Gherkin scenarios from BPMN process models using transformation patterns and templates. A tool was developed to automate this process, validated through qualitative expert evaluation, confirming its utility and accuracy. The approach enhances organizational communication and collaboration between business and information technology teams, improving efficiency in requirements elicitation. Future enhancements aim to broaden transformation patterns and tool functionalities to encompass additional BPMN artifacts. This study emphasizes innovation in bridging business process modeling and agile development, highlighting advancements in automating requirements elicitation.

Keywords: BPMN; user stories; Gherkin scenarios; requirements engineering; agile development; business process modeling; automation; transformation patterns



Academic Editors: Małgorzata B Pankowska and Dominik Spinczyk

Received: 16 July 2025

Revised: 22 August 2025

Accepted: 26 August 2025

Published: 28 August 2025

Citation: Mateus, D.; da Silveira, D.S.; Araujo, J. Pattern-Based Automation of User Stories and Gherkin Scenarios from BPMN Models for Agile Requirement. *Appl. Sci.* **2025**, *15*, 9434. <https://doi.org/10.3390/app15179434>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Business process modeling is a cornerstone for organizations that seek to understand and optimize operational workflows, improving efficiency and organizational performance [1]. As business processes become more dependent on information systems, bridging the communication gap between business analysts and technical developers is increasingly important to ensure that system implementations reflect organizational needs [2].

This study focuses on the elicitation of functional requirements within agile development contexts, aiming to improve communication between business and information technology stakeholders. Specifically, we propose a systematic method to extract two complementary types of requirements artifacts from business processes models: user stories, which capture functionality from the user perspective, and Gherkin scenarios, which describe concrete behavioral examples using the Given-When-Then template [3,4]. While user stories express who needs what and why, Gherkin scenarios provide executable acceptance criteria that are directly useful in behavior-driven development (BDD) and automated testing.

Despite prior advances in using process models to inform requirements, there remains a gap in methods that reliably and consistently transform BPMN models into both user stories and Gherkin scenarios. To address this gap, we introduce a pattern-driven transformation approach grounded in partial metamodels of BPMN, user stories, and Gherkin scenarios. Transformation templates map BPMN constructs (e.g., tasks, events, gateways) into parametrized natural-language artifacts, while a glossary-based mechanism helps disambiguate domain labels to preserve semantics during generation [5].

In addition, this paper presents a web-based support tool and a qualitative validation conducted with domain experts. The proposed solution aims to generate, directly from process models, user stories and scenario descriptions that are clear, consistent, and sustainable, thereby strengthening traceability between business specifications and acceptance criteria in agile development. The control activities implemented to enhance transparency and ensure the reproducibility of the transformation process are also detailed (see Section 7).

Unlike prior approaches, such as those by Leopold et al. [6] and Aysolmaz et al. [7], which emphasize semi-automatic generation or template-dependent document production, our method uses reusable transformation patterns to generate both user stories and executable Gherkin scenarios in an integrated process. This unified output supports agile practices by coupling functional requirement specification with scenario-based acceptance criteria, thereby improving traceability from BPMN elements to executable tests and facilitating collaboration between business and development teams.

The remainder of the paper is organized as follows. Section 2 presents background on BPMN, requirements engineering, and agile development. Section 3 discusses related work and positions our contribution relative to existing methods. Section 4 introduces the formalized transformation patterns. Section 5 describes the tool that supports the approach. Section 6 reports the evaluation with experts and discusses results. Finally, Section 7 concludes and outlines avenues for future work. To operationalize this proposal, the next section presents the methodological foundations and the concepts necessary to support the transformation process.

2. Materials and Methods

This section provides an overview of the core foundations underpinning our study, focusing on BPMN, requirements engineering, and agile development, with emphasis on user stories and Gherkin scenarios.

2.1. Business Process Modeling Notation

BPMN plays a pivotal role in visualizing and analyzing organizational workflows, supporting a wide range of stakeholders, including business analysts, technical developers and executives [3]. Designed for comprehensibility across user groups, BPMN addresses the needs of (i) business analysts, who create initial process documentation, (ii) developers, who implement processes, and (iii) executives, who oversee and monitor execution [2].

Maintained by the Object Management Group (OMG), BPMN consolidates elements from pre-existing notations such as the XML Process Definition Language (XPDL) [8] and UML Activity Diagrams [9]. This consolidation has enhanced BPMN's versatility and facilitated its broad adoption across organizational contexts.

At its foundation, BPMN operates within a metamodel framework, itself grounded in the Unified Modeling Language (UML) [9]. The metamodel is structured across layers, with the *Core* layer being central, encompassing essential elements for building BPMN diagrams [5]. These include *Processes*, *Choreographies*, and *Collaborations*, each defining different aspects of process representation.

Of particular relevance to this study is the process metamodel, illustrated in Figure 1 [5], which serves as the basis for the transformation patterns. The fragment highlights the hierarchical relationships among BPMN components. For instance, the metaclass *Process* inherits from *FlowElementsContainer*, which contains lanes and flow elements that, in turn, include *FlowNodes* such as *Activities*, *Events*, and *Gateways*. These are connected by *SequenceFlows* that establish control and communication paths within a process. *Gateways* are specialized into exclusive, inclusive, parallel, and event-based types, while *Events* are categorized as start, intermediate, or end. By grounding our transformation patterns in these fundamental BPMN elements, we enable the systematic generation of user stories and Gherkin scenarios, fostering synergy between business process modeling and agile development practices.

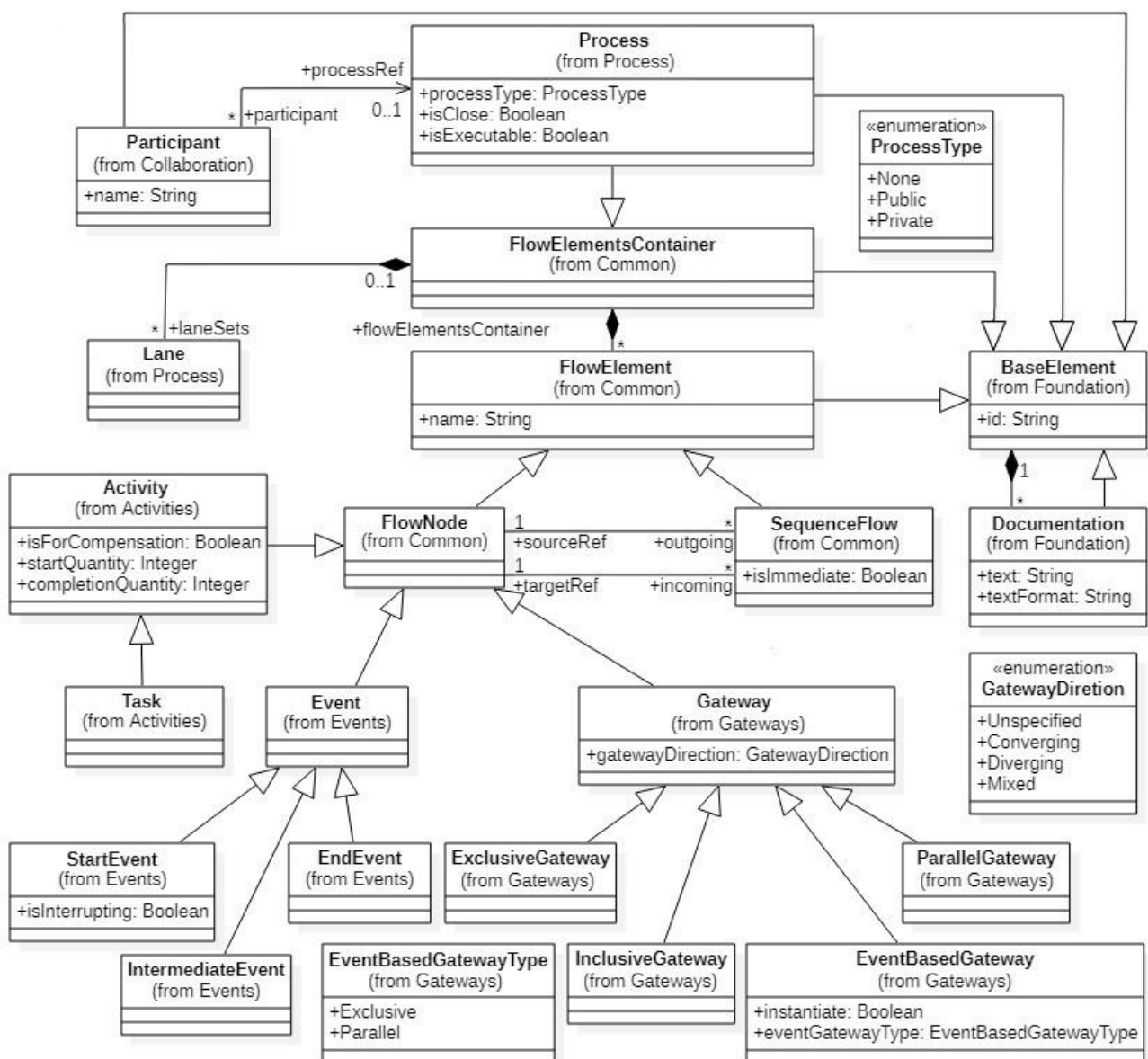


Figure 1. Fragment of the BPMN metamodel [5].

2.2. Requirements Engineering and Agile Development

Requirements engineering (RE) is the process of identifying and specifying the services expected from a system, along with the constraints governing its operation and development [10]. At its core, RE aims to articulate solutions to underlying problems in

a structured manner. Sommerville’s spiral model emphasizes the iterative nature of RE through elicitation, specification, validation, and management phases [10]. This study focuses specifically on elicitation and specification, which must often be adapted to the context of the process, project, product, and stakeholders involved.

Among development methodologies, agile approaches stand out for their incremental and adaptive nature, emphasizing collaboration and responsiveness to evolving requirements [11]. Within this paradigm, requirements are captured through lightweight artifacts such as user stories and scenarios, consistent with the principles of behavior-driven development (BDD).

User stories provide concise narratives of end-user requirements, typically following the template [12]:

As a “type of user” (Role or type of user), I want to “desired goal” (The need/functionality that the user intends to see fulfilled) so that “achieved value” (Purpose of the functionality in question).

BDD complements this approach by bridging technical and business perspectives through scenario-based specifications. Scenarios, often written in Gherkin syntax, describe expected system behaviors in the structured “Given-When-Then” format [4]:

- *Given*: Describes the scenario’s initial context.
For example, given my account has a balance of EUR 430;
- *When*: Describes an event or action.
For example, when I send the money;
- *Then*: Describes the expected result.
For example, then, I should get the receipt.

Figure 2 presents a metamodel that organizes the elements of user stories and Gherkin scenarios and their relationships. Each user story may be linked to multiple scenarios, while stories and scenarios themselves are structures through components, such as *Role*, *Goal*, and *Benefit* (for stories), and *Given*, *When*, and *Then* (for scenarios). This metamodel provides the blueprint for integrating user stories and scenarios into the software development lifecycle. The transformation rules applied in this study are grounded in this metamodel and are detailed in Section 4.1 (*Formalization of Transformation*), where each BPMN construct is mapped to corresponding elements of user stories and scenarios.

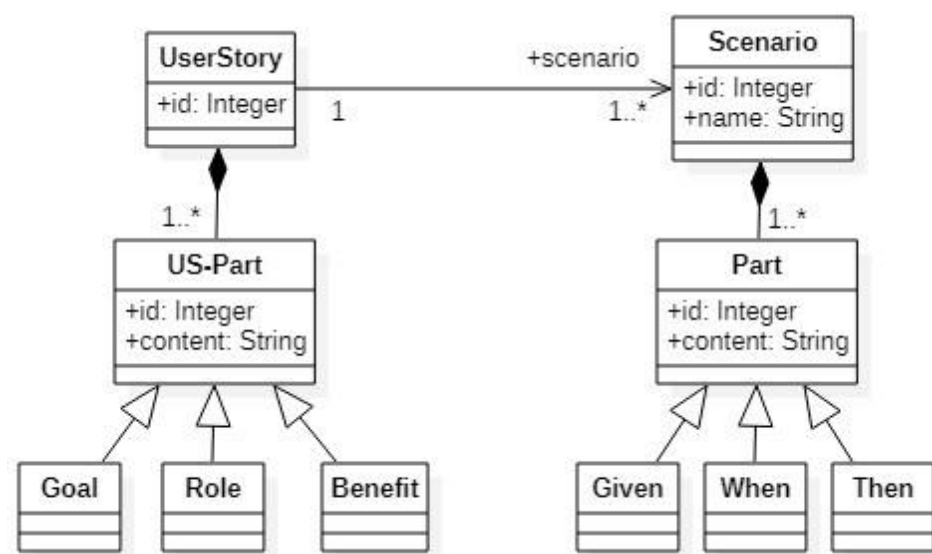


Figure 2. User story and Gherkin scenario metamodel.

Recent advances in RE have explored techniques, such as Natural Language Processing (NLP), Model-Driven Engineering (MDE), and Artificial Intelligence (AI), to automate the extraction and refinement of requirements. A recent systematic review highlights growing efforts to integrate NLP and AI into RE, enhancing automation and precision [13]. Similarly, MDE approaches have been applied to manage BPMN model families through metamodel-based transformations and tool support [14]. However, these approaches often focus either on generating isolated textual requirements or on model transformations without systematizing producing behavior-driven development (BDD) scenarios, particularly in Gherkin syntax. Moreover, they frequently lack mechanisms to ensure traceability between BPMN constructs and generated artifacts or require extensive manual template adaptation for specific domains.

Our approach addresses these limitations by combining BPMN-based modeling with pattern-driven transformations to automatically generate both user stories and executable Gherkin scenarios in a unified and semantically consistent manner. This integration enhances precision, traceability, and adaptability within agile contexts.

3. Related Work

Business process modeling has long been studied as a foundation for software systems requirements specification. A wide range of studies emphasizes its central role in requirements engineering, supporting both manual and automated approaches for eliciting, analyzing, and validating requirements. Below, we synthesize the main contributions in the field, focusing on how process models have been leveraged to improve requirements elicitation and automation.

Manual approaches to using process models in requirements engineering can be broadly categorized into two groups: (i) those that derive textual requirements directly from process models and (ii) those that generate model-based requirements. For example, Cardoso et al. [15] analyze the automation potential of process activities and produce corresponding textual requirements, a strategy also adopted by Ma and Jiang [16]. Mayr et al. [17] map detailed textual requirements to process model activities, while Li et al. [18] link requirements to model elements to uncover dependencies and detect ambiguities or omissions. Demirörs et al. [19] extend this idea to cover non-functional, security, and hardware requirements, and Monsalve et al. [20] examine process modeling notations for eliciting and representing user requirements at a strategic level. These works illustrate how process models can help identify requirement gaps, validate specifications with stakeholders, and ensure alignment with business goals.

Other studies focus on enriching process models to derive additional artifacts. González and Díaz [21] propose deriving goal models from process activities, which are then used to establish use cases and their relationships. Similarly, Cox et al. [22] define a method for manually constructing problem frame diagrams alongside textual requirements using functional activity diagrams. While these methods improve the expressiveness of process models for requirements elicitation, they lack automated support.

In contrast, automated approaches treat process models as the basis for generating requirements artifacts, often producing natural language descriptions to aid elicitation and validation. Leopold et al. [6] analyze activity labels and control flow to automatically generate textual descriptions of process models. Malik and Bajwa [23] employ a template-based sentence generation algorithm for requirements, while Türetken et al. [24] broaden the range of process elements considered, including roles, data inputs/outputs, events, and systems. Coşkunçay et al. [25] highlight the need for additional models to capture data required for process automation, though their approach lacks a detailed requirements analysis method and a formal generation technique. These studies collectively underscore

the potential of automating requirements specification from process models, yet most remain limited to textual outputs.

Some works address more specific aspects. Cardoso et al. [15] compare the coherence and traceability of requirements derived from business process models with those from traditional approaches. Herden et al. [26] explore BPMN as an alternative to textual use cases or UML activity diagrams, showcasing BPMN's versatility in requirements engineering. Aysolmaz et al. [7] present a semi-automated approach for generating requirements documents from Event Driven Process Chain (EPC) models using template-based natural language generation, enabling systematic knowledge transfer from process models to other software lifecycle activities.

However, existing solutions often provide only partial automation. For example, Leopold et al. [6] and Malik and Bajwa [23] focus on producing textual descriptions or business rules but do not address agile artifacts such as user stories or behavior-driven development (BDD) scenarios. Similarly, modeling frameworks, like BORE, e³ value [27], and i* [28], emphasize business goals and value exchanges but lack both automation mechanisms and formal pattern-based transformation logic applicable in agile contexts.

Recent research has begun to explore novel directions for automation. Large Language Models (LLMs) have been applied to generate BPMN models from textual descriptions [29], while the vision of AI-augmented Business Process Management systems [30] suggests new opportunities for integrating reasoning and automation in process modeling. Likewise, recent surveys on NLP for requirements engineering [13] highlight advancements in semantic parsing, domain adaptation, and automated artifact generation. Although promising, these works do not address the direct generation of Gherkin-based scenarios from BPMN nor do they fully integrate semantic traceability between process models and agile artifacts.

Our proposal differs in that it introduces a systematic, tool-supported methodology for pattern-driven generation of both user stories and Gherkin scenarios directly from BPMN models. The main contributions include: (i) a formally defined set of transformation patterns linking BPMN constructs to agile textual artifacts; (ii) automation of requirements extraction while preserving process model semantics; (iii) an extensible interface allowing the creation of domain-specific patterns; and (iv) glossary-based disambiguation for handling specialized terminology.

Compared to label- or template-driven approaches [6,7], our method explicitly targets scenario-based artifacts (Gherkin) while preserving semantic traceability from BPMN constructs to text through structured pattern tuples. It also supports extensibility via a configurable glossary and user-defined patterns, addressing three major limitations in the literature: (i) scalability to large and complex BPMN diagrams, (ii) adaptability to diverse modeling styles and domain-specific vocabularies, and (iii) coverage of exception and alternative flows in addition to main flows.

Importantly, the joint generation of user stories and Gherkin scenarios represents more than an incremental improvement. In practical agile workflows, this integration reduces the gap between requirements documentation and acceptance testing, minimizing translation errors and ensuring that business goals are immediately reflected in executable validation artifacts. By enabling direct incorporation of generated stories and scenarios into agile toolchains, the approach fosters traceability across the software lifecycle, shortens feedback loops in sprint cycles, and mitigates the risk of misalignment between business analysts and development teams. This practical impact highlights how the proposed integration contributes not only methodologically but also operationally to agile practices.

In summary, while existing approaches have advanced automated requirements generation from BPMN models, they typically focus on textual requirements or rely heavily on

static templates. Our approach extends these contributions by providing a reusable, pattern-based framework that simultaneously produces user stories and executable Gherkin scenarios, integrating functional requirements documentation with scenario-based acceptance criteria in a unified process. This structured integration of agile artifacts, semantic traceability, and extensible automation positions our contribution as both timely and well-aligned with recent advances in AI-driven and NLP-assisted requirements engineering [13,29,30].

To synthesize the main distinctions among existing approaches, Table 1 provides a comparative overview. The table highlights the types of input artifacts, the resulting outputs, the degree of automation, and the level of traceability support. This comparison underscores how our approach advances the state of the art by combining BPMN-based transformations with the automated generation of both user stories and executable Gherkin scenarios, while ensuring explicit traceability.

Table 1. Comparison of related approaches.

Authors/Ref.	Input Artifacts	Output Artifacts	Automation Level	Traceability Support
Cardoso et al. [15]	BPMN process models (activity and control-flow)	Natural language textual descriptions of processes	Semi-automated	Limited (implicit links between activities and text)
Ma & Jiang [16]	BPMN models and domain-specific patterns	Requirement specifications	Semi-automated	Partial (pattern-based associations)
Mayr et al. [17]	BPMN models enriched with contextual data	Use cases and requirement statements	Automated	Partial (some element-to-artifact mapping)
Li et al. [18]	BPMN models	User stories (textual form)	Semi-automated	Limited (manual refinement needed)
Demirörs et al. [19]	BPMN and process-related documentation	Requirements and validation artifacts	Automated	Explicit (formal mappings defined)

4. Transformation Patterns

Business process models serve as blueprints that depict the sequence of organizational activities. These models are built using domain-specific terminology, encapsulating key process elements, such as *Activities*, *Events*, and *Gateways*, in the BPMN notation. In agile software development, these BPMN diagrams play a pivotal role by enabling software engineers to interpret business logic and translate it into user stories that articulate functional requirements.

4.1. Formalization of Transformation

The transformation patterns proposed in this study were systematically derived through the analysis of the syntactic and semantic properties of BPMN elements, based on the BPMN 2.0 metamodel. A rule-based methodology was adopted to define each transformation rule, mapping BPMN flow nodes to natural language templates used in user stories and Gherkin scenarios. This ensures that the transformed textual artifacts faithfully preserve the structural and semantic meaning of the original process models.

Each transformation pattern comprises of three core components:

- **Source construct:** The BPMN element or combination of elements that trigger the transformation, such as a Task followed by an *Exclusive Gateway*.

- **Preconditions and contextual rules:** Constraints or conditions required for the transformation to be valid (e.g., type of gateway, flow labels, or the existence of a defined role).
- **Target template:** A parameterized natural language structure representing the resulting user story or Gherkin scenario (e.g., “As a [role], I want to [action], so that [goal]” or “Given [context], When [event], Then [outcome]”).

We intentionally adopted a semi-formal, pattern-based representation for the transformation rules rather than a fully formal model-transformation notation (e.g., ATL/QVT). This design choice was driven by the primary objective of maximizing accessibility and practical adoption across multidisciplinary teams, particularly business analysts and process managers who frequently author BPMN models but are not familiar with formal transformation languages. While a highly formal specification increases mathematical rigor, it can also raise the barrier for practitioners to understand, extend, and reuse the patterns in everyday settings.

At the same time, we recognize the importance of repeatability and traceability. To balance these needs, each pattern in our approach is specified in a lightweight, structured form (readable tuple + parameterized template) that preserves unambiguous mapping from BPMN constructs to textual artifacts while remaining editable through the tool’s user interface. This semi-formal compromise ensures methodological transparency and supports future formalization efforts (e.g., ATL/QVT encodings) without hampering immediate usability in organizational contexts.

This formalization promotes traceability between process model elements and their corresponding textual artifacts. For example, a Task associated with a Lane or Pool provides the “role” component in a user story, while control paths defined by gateways determine the conditions for scenario generation.

To ensure syntactic integrity, the tool validates BPMN model compliance prior to transformation, verifying elements such as sequence flow connections and event configurations. In terms of semantic clarity, the transformation relies on a glossary of terms to disambiguate labels, allowing consistent differentiation between natural and alternative flows. Within this context, the glossary serves as a controlled vocabulary, enabling the disambiguation of activity and role names, as well as the differentiation between natural and alternative flows. Glossary entries consist of terms, definitions, and optional synonyms, which can be aligned with domain-specific vocabularies. The glossary is stored in a structured JSON format, allowing portability between projects and ensuring reproducibility of transformations. During execution, each BPMN label is cross-referenced against the glossary to standardize terminology, thereby reducing variability in the generated user stories and Gherkin scenarios and increasing traceability across different models and domains.

Although the initial pattern set was derived from a limited number of models, the approach is extensible. The tool allows the definition of custom transformation rules, facilitating adaptation to domain-specific vocabularies and templates. Future work includes the formal specification of these transformations using model transformation languages (e.g., ATL or QVT), enabling automated verification and refinement.

In addition to the predefined transformation patterns, the tool provides a user-friendly graphical interface that enables users to define and register new transformation rules. This extension mechanism is particularly valuable for tailoring the tool to different modeling practices and domain-specific semantics. Users can associate selected BPMN constructs with customized user story or scenario templates, define preconditions, and preview transformation outcomes directly within the interface. This built-in extensibility ensures that the transformation logic remains adaptable, scalable, and aligned with the evolving needs of organizations.

Furthermore, while the present work deliberately adopts a semi-formal structure to maximize accessibility and adoption by practitioners, we recognize the importance of advancing towards more rigorous formal specifications. In future work, the transformation patterns could be encoded in established model transformation languages such as ATL (ATLAS Transformation Language) or QVT (Query/View/Transformation). These languages would enable automated verification, formal reasoning over transformation rules, and enhanced interoperability with model-driven engineering environments, thereby complementing the usability-oriented approach adopted in this study.

4.2. Extraction of Patterns

The extraction of transformation patterns from BPMN models is both feasible and essential. Below we detail a non-exhaustive list of patterns identified through the analysis of a variety of business process models. Each pattern enables the extraction of user stories from distinct modeling configurations:

1. *StartEvent* followed by an *Activity*:
Whenever a *SequenceFlow* is found with a *StartEvent* at its source (+sourceRef) and its target (+targetRef) with an *Activity* in a *Lane*, the corresponding user story will follow the following format:
US1. As a(n) "*Lane*", I receive a(n) "*StartEvent*" in order to execute the "*Activity*"
2. *MessageStartEvent* connected to a *Participant* and a *MessageEndEvent* in different or the same *Lanes*:
Whenever a *MessageFlow* is found with a *Participant* at its source (+sourceRef) and its target (+targetRef) a *StartEvent* and an *EndEvent* of the type "*Message*", the corresponding user story will follow this format:
US2. As a "*Participant*", I want to send "*StartEvent*" in order to get "*EndEvent*"
3. *Activity*₁ followed by *Activity*₂:
Whenever a *SequenceFlow* is found with an *Activity*₁ at its source (+sourceRef) and its target (+targetRef) includes another *Activity*₂ within a *Lane*, the corresponding user story will follow this format:
US3. As a "*Lane*", I execute "*Activity*₁" to subsequently execute "*Activity*₂"
4. *Activity* followed by a ("throw") *MessageIntermediateEvent*:
Whenever a *SequenceFlow* is found with an *Activity* at its source (+sourceRef) and an *IntermediateEvent* of the type "*Message*" ("throw") at its target (+targetRef), and this *IntermediateEvent* acts as the source (+sourceRef) for a *MessageFlow* directed towards a *Participant* as its target (+targetRef) (representing an external actor in the process), the corresponding user story will follow the following format:
US4. As a(n) "*Lane*", I execute "*Activity*" in order to send a(n) "*IntermediateEvent*" to "*Participant*"
5. *Activity* followed by a ("catch") *TimerIntermediateEvent* followed by a *MessageEndEvent*:
Whenever a *SequenceFlow* is found with an *Activity* at its source (+sourceRef) within a *Lane*, and its target (+targetRef) is an *IntermediateEvent* of the type "*Timer*", and another *SequenceFlow* is identified with the same *IntermediateEvent* at its source (+sourceRef), and its target (+targetRef) is an *EndEvent* of the type "*Message*", the corresponding user story will follow the following format:
US5. As a "*Lane*", I execute "*Activity*", then I wait for "*IntermediateEvent*" in order to send "*EndEvent*" to "*Participant*"
6. *Activity* followed by a ("throw") *MessageEndEvent* in different or same *Lanes*:
Whenever a *SequenceFlow* is found with an *Activity* at its source (+sourceRef) within a *Lane*, and its target (+targetRef) is an *EndEvent* of the type "*Message*" located in a different or same *Lane* than the *Activity*, and a corresponding *MessageFlow* is identified

with the same *EndEvent* at its source (+sourceRef) and its target (+targetRef) is a *Participant*, the corresponding user story will follow the following format:

US6. As “Lane₁” I execute “Activity” in order for “Lane₂” to send “EndEvent” to “Participant”

7. ExclusiveGateway join or split between two Activities in different or same Lanes:
Whenever a *SequenceFlow* is encountered with an *Activity* at its source (+sourceRef) and its target (+targetRef) is an *ExclusiveGateway* join or split, and the *SequenceFlow* from this *ExclusiveGateway* join or split has another *Activity* as its target (+targetRef) in a different *Lane* from the previous one, the corresponding user story will follow the following format:
US7. As “Lane₁” I want to “Activity₁” in order to “Lane₂” can “Activity₂”
8. ExclusiveGateway join between a (“catch”) *MessageStartEvent* and an *Activity*:
Whenever a *SequenceFlow* is found with a *StartEvent* of the type “Message” at its source (+sourceRef), and its target (+targetRef) is an *ExclusiveGateway*—which, in turn, serves as the source (+sourceRef) for another *SequenceFlow* that has an *Activity* as its target (+targetRef) within a *Lane*—the corresponding user story takes the following form:
US8. As “Lane”, I receive a(n) “StartEvent” to subsequently execute “Activity”
9. ExclusiveGateway join between a (“catch”) *TimerIntermediateEvent* and an *Activity*:
Whenever a *SequenceFlow* is found with a *IntermediateEvent* of the type “Timer” at its source (+sourceRef), and its target (+targetRef) is an *ExclusiveGateway*—which, in turn, serves as the source (+sourceRef) for another *SequenceFlow* that has an *Activity* as its target (+targetRef) within a *Lane*—the corresponding user story takes the following form:
US9. As “Lane”, I want to wait for “IntermediateEvent” to subsequently execute “Activity”
10. *InclusiveGateway* split connected with more than one *Activity*:
Whenever more than one *SequenceFlow* is found with the same *InclusiveGateway* at its source (+sourceRef), and their respective targets (+targetRef) are different *Activities* within *Lanes*, each *Lane* will have a corresponding user story for its respective *Activities*, following this format, if the condition associated with the corresponding *SequenceFlow* is validated as true:
US10. As “Lane”, I want to execute one or more “Activity_{1..n}” together depending on the respective “Condition”
11. *InclusiveGateway* join between a *MessageStartEvent* and an *Activity*:
Whenever a *SequenceFlow* is found with a *StartEvent* of the type “Message” at its source (+sourceRef), and its target (+targetRef) is an *InclusiveGateway*—which, in turn, serves as the source (+sourceRef) for another *SequenceFlow* that has an *Activity* as its target (+targetRef) within a *Lane*—the corresponding user story takes the following form:
US11. As “Lane”, I receive a(n) “StartEvent” to subsequently execute “Activity”
12. *ParallelGateway* split connected with “n” *Activities*:
Whenever more than one *SequenceFlow* is found with the same *ParallelGateway* at its source (+sourceRef), and their respective targets (+targetRef) are different *Activities* within *Lanes*, each *Lane* will have a corresponding user story for its respective *Activities*. The story follows this format, with these *Activities* always being carried out together:
US12. As “Lane”, I want to execute “Activity_{1..n}” together;
13. *TimerIntermediateEvent* followed by an *Activity*:
Whenever a *SequenceFlow* is found with an *IntermediateEvent* of the type “Timer”, and its target (+targetRef) is an *Activity* within a *Lane*, the corresponding user story will follow the following format:
US13. As “Lane”, I want to wait for “IntermediateEvent” to subsequently execute “Activity”.

4.3. Applying the Patterns

To assess the effectiveness of the proposed patterns, we applied them to a BPMN model representing a real estate assessment process (Figure 3). This is a simple example that starts with the registration of the documentation followed by the checking of this documentation. If the documentation is okay, the evaluation is scheduled; otherwise, it must be fixed. After the notification of the evaluation date is produced, the property is evaluated, and a suggested value is sent to the owner. The following user stories were automatically extracted:

1. *StartEvent* followed by an *Activity*:
US: As a *Customer Service*, I receive an *Evaluation request* in order the *Register Documentation*.
2. *ExclusiveGateway* join or split between two *Activities* in different or same *Lanes*:
US: As *Customer Service* I want to *Register Documentation* in order to *Judicial* can *Check Documentation*.
3. *ExclusiveGateway* join or split between two *Activities* in different or same *Lanes*:
US: As *Judicial* I want to *Check Documentation* in order to *Customer Service* can *Schedule Evaluation*.
4. *Activity* followed by a (“throw”) *MessageIntermediateEvent*:
US: As a *Customer Service*, I execute *Schedule Evaluation* in order to send a *Notification of Evaluation Date* to *Owner*.
5. *ExclusiveGateway* join between a (“catch”) *MessageStartEvent* and an *Activity*:
US: As *Judicial* I receive a *Regularization of Documentation* to subsequently execute *Check Documentation*.
6. *TimerIntermediateEvent* followed by an *Activity*:
US: As *Brokerage*, I want to wait for *Date and Time of the Evaluation* to subsequently execute *Evaluate Property*.
7. *Activity*₁ followed by *Activity*₂:
US: As *Brokerage*, I execute *Evaluate Property* to subsequently execute *Suggest Value*.
8. *MessageStartEvent* connected to a *Participant* and a *MessageEndEvent*:
US: As *Owner*, I want to send *Evaluation Request* in order to get *Property (Evaluated)*.
9. *Activity* followed by a *MessageEndEvent* in different or the same *Lanes*:
US: As *Brokerage* I execute *Suggest Value* in order to for *Brokerage* to send *Property (Evaluated)* to *Owner*.

While the extracted stories effectively capture the natural flow of the process, alternative or exceptional paths also exist. For instance, the “*Check Documentation*” task may lead to an irregularity, which triggers an alternate scenario.

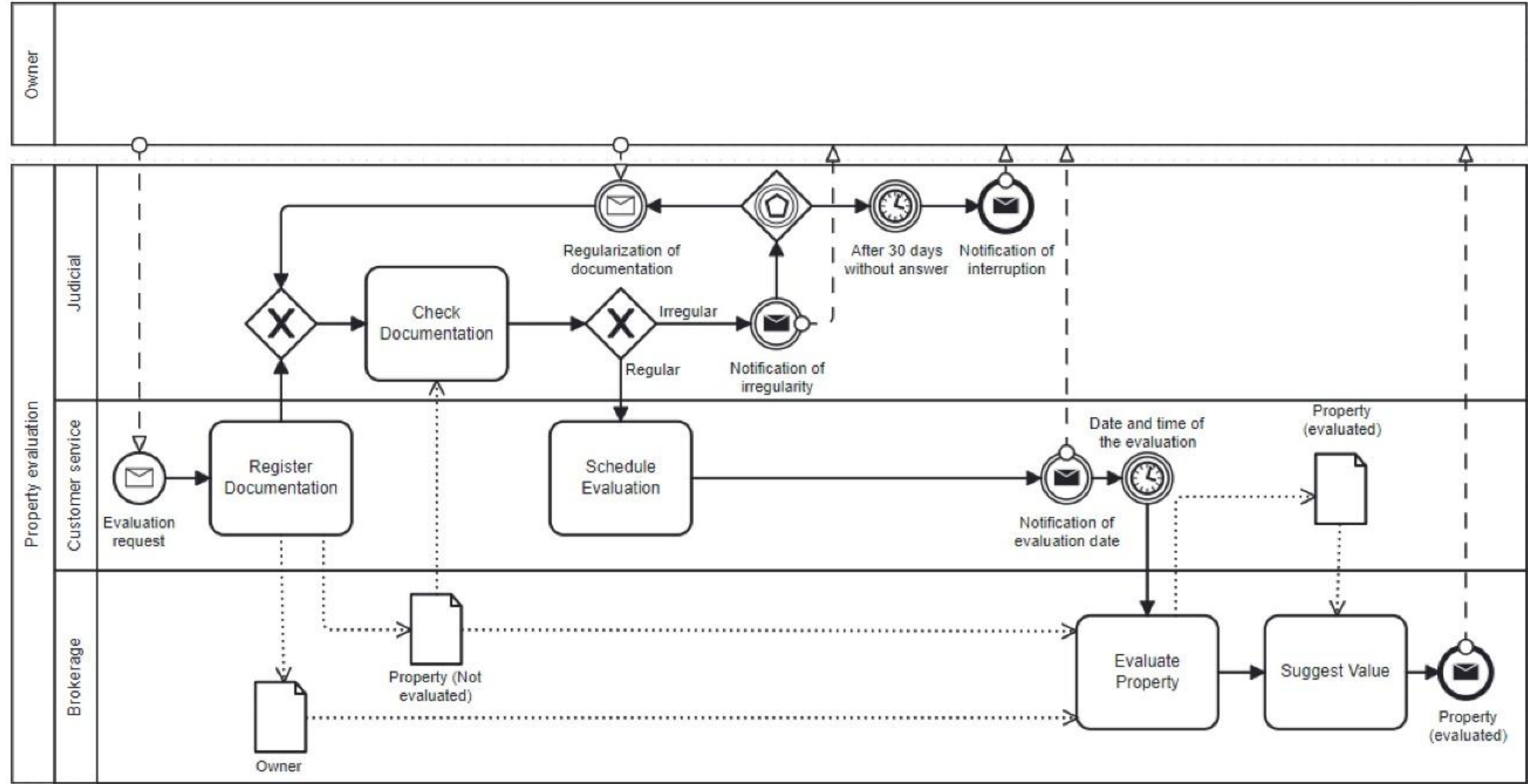


Figure 3. BPMN model (property evaluation).

4.4. Scenario Integration

After extracting the user stories, alternative or exceptional paths must be identified and described using Gherkin syntax. Figure 3 provides the BPMN model from which the following Gherkin scenarios were derived:

Feature: “Property Evaluation”

1. # Check Documentation Irregular Path

Scenario: “Judicial” “Check Documentation” as “Irregular”;

Given “Check Documentation” as “Irregular” by “Judicial”;

When “Check Documentation” encounters “Irregularity”;

Then “Judicial” sends a “Notification of Irregularity” to the “Owner”.

2. # 30 Days without Answer

Scenario: “Judicial” “Notification of irregularity” “After 30 days without answer”;

Given “Judicial” send “Notification of irregularity”;

When “After 30 days without answer”;

Then “Judicial” sends “Notification of interruption”.

5. Transformation Tool

This section presents the transformation tool developed to automate the generation of user stories and Gherkin scenarios from BPMN process models. The tool implements the transformation patterns introduced in this study, enabling the extraction of both artifacts based on the structure and flow logic of the process models.

During the tool’s initial development phase, a key consideration was selecting a platform that would allow seamless integration with a business process modeling environment. After evaluating various alternatives, we chose BPMN.io [31], a web-based modeling tool that offers native support for BPMN 2.0 and provides extensive diagramming capabilities through a modern, open-source interface.

Once a process model is created using BPMN.io, it can be exported as a “.bpmn” file, an XML-based representation of the model. Our tool reads and processes this file using the *System.Xml* namespace from the Microsoft .NET framework, enabling extraction, interpretation, and transformation of the model elements into user stories and Gherkin scenarios.

5.1. Solution Specification

The core function on the transformation tool is to systematically interpret the *Flow Nodes* within the BPMN process model. *Flow Nodes* may include *Events*, *Gateways*, and *Activities* (as illustrated in Figure 1).

The tool analyzes both natural and alternative flows. Natural flows represent the standard execution path of a process and are mapped to user stories. Alternative flows, often arising from conditional or exceptional paths, are transformed into Gherkin scenarios. To distinguish between these flows, the tool relies on a configurable glossary of terms, as conceptually detailed in Section 4.1, to ensure consistent and unambiguous mapping of BPMN element labels to user stories and Gherkin scenarios.

For example, Figure 4 highlights a fragment of the BPMN model shown in Figure 3. After the “Check Documentation” activity, the exclusive gateway directs the flow to either a “Irregular” and “Regular” path. Based on terminology defined in the glossary, the tool determines which path corresponds to the natural flow and which to an alternative one.

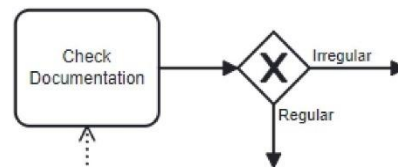


Figure 4. Fragment of the property evaluation model.

All transformation patterns, including both predefined and custom ones, are stored within the tool and can be managed through the “Patterns” menu. As illustrated in Figure 5, users can access the “Add New Patterns” feature to define new templates that accommodate organizational differences in user story or scenario formats. The prototype includes an initial set of patterns, and users are encouraged to expand this collection to better align with domain-specific requirements.

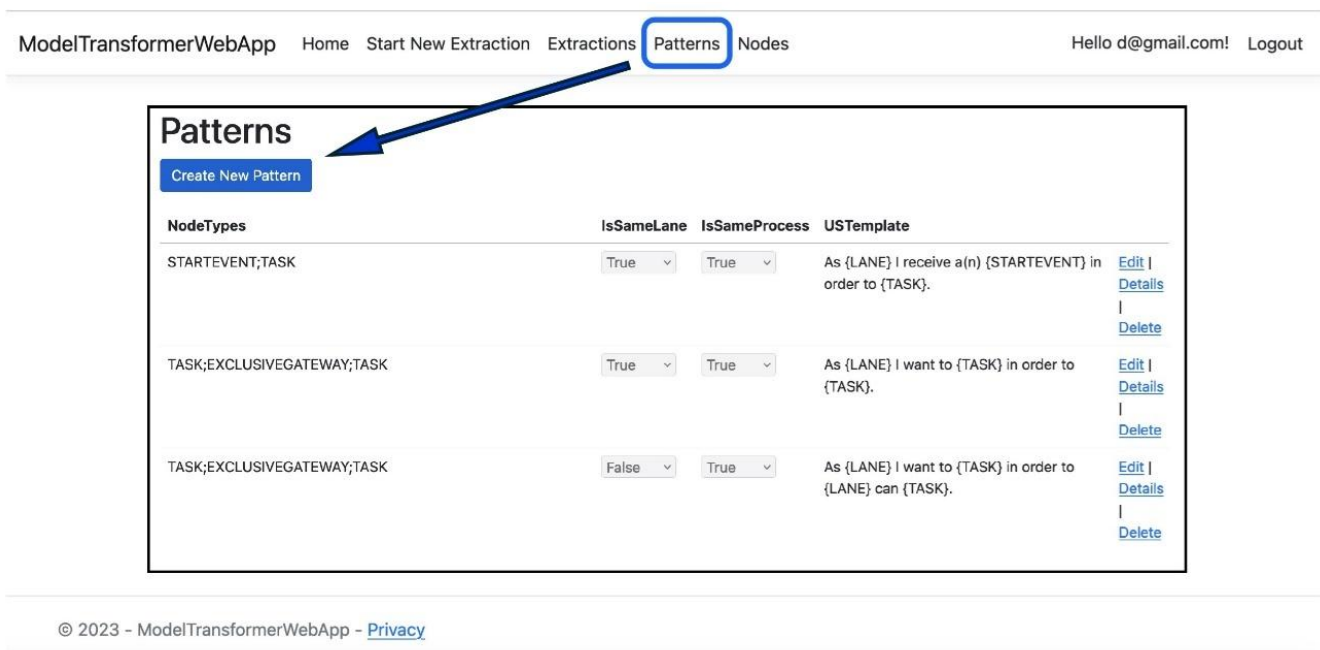


Figure 5. Creating new pattern templates in the tool.

Once the flow paths and associated nodes are identified, the tool applies the transformation patterns to generate the respective user stories and Gherkin scenarios.

In addition to pattern configuration, the tool’s interface includes the following core functionalities accessible from the top menu:

- *Start New Extraction*: Opens the main workspace for uploading and processing BPMN files.
- *Extractions*: Provides access to the history of prior extractions and related metadata.
- *Nodes*: Displays the list of flow nodes identified from previously analyzed models.

Currently, the tool supports a subset of BPMN elements commonly found in basic process models, including tasks, exclusive and parallel gateways, start and end events, sequence flows, and message flows. However, more advanced elements, such as subprocesses, event-based gateways, escalation events, data objects, and artifacts, are not yet supported. This limitation affects the applicability of the tool in complex enterprise scenarios. Nevertheless, the tool was designed with a modular and extensible architecture, which facilitates the future incorporation of these constructs and their corresponding transformation patterns.

Furthermore, the tool currently focuses on generating Gherkin scenarios based on alternative flows derived from decision points within the BPMN models. This design decision was motivated by the assumption that alternative flows often represent exceptional or edge-case behavior, which typically requires more detailed testing and clarification. However, the tool does not yet support the generation of scenarios for main success flows, which are equally critical for defining acceptance criteria in agile development. We recognize this as a limitation. The tool's architecture allows for the future integration of mechanisms to identify and process main success paths, either through predefined rules or annotations within the BPMN model. Future work will prioritize the implementation of this feature to ensure broader applicability of the tool in real-world agile scenarios.

5.2. Tool Architecture and Integration with BPMN.io

The prototype tool was implemented as a web-based application to maximize accessibility and ease of deployment across different organizational environments. Its architecture follows a modular design, composed of four main layers:

1. **Presentation Layer:** developed in JavaScript and HTML5, incorporating the BPMN.io library for BPMN model creation, visualization, and editing directly within the browser. This layer allows users to either design new process models from scratch or import existing BPMN 2.0 XML files.
2. **Transformation Layer:** responsible for applying the predefined transformation patterns to the BPMN model. Implemented in JavaScript, it parses the BPMN 2.0 XML representation, identifies relevant constructs, validates them against preconditions, and maps them to the corresponding user story or Gherkin scenario templates.
3. **Pattern Management Layer:** manages the storage and retrieval of transformation patterns. Patterns are stored in a JSON-based repository, which allows both predefined and custom user-defined rules to be loaded, modified, or extended without altering the core codebase.
4. **Persistence and Export Layer:** handles the saving of customized patterns and the export of generated artifacts (user stories and Gherkin scenarios) in structured text formats. This ensures that generated requirements can be easily integrated into agile project management tools or documentation repositories.

The BPMN.io integration is central to the tool's operation. BPMN.io's open-source modeling capabilities are leveraged to access the full BPMN 2.0 metamodel in XML form, enabling accurate identification of flow nodes, gateways, events, and their interconnections. Its API allows real-time interaction between the modeler and the transformation engine, so that any model changes are immediately reflected in the generated requirements. This tight integration ensures semantic consistency between the visual process model and the textual artifacts produced.

The modular architecture also supports scalability and maintainability, allowing future extensions such as the integration of natural language processing components for more sophisticated label interpretation or connectors to external requirement management systems. While the current prototype is optimized for small-to-medium process models, the architecture provides a foundation for handling more complex processes through incremental optimizations.

6. Evaluation of the Approach

The evaluation of this transformation approach begins with a coverage analysis performed by the authors, followed by a questionnaire in which a group of five information technology experts assessed the extraction of user stories and Gherkin scenarios from some

BPMN business process models. The following subsections describe in some detail the findings of this evaluation.

6.1. Solution Specification

This section aims to conduct a coverage analysis with the objective of identifying both the topics addressed and those not addressed in the development of this approach.

6.1.1. Patterns Coverage

The transformation patterns, as detailed in Section 4.1, is only an initial set of patterns, and as mentioned earlier, more are expected to be added as the approach is used further. Therefore, if a model requires a pattern that has not yet been identified, the content associated with that pattern will not be automatically extracted. However, to address this issue, the designer should add the presumed pattern to the transformation prototype.

In this sense, some new patterns, in addition to those mentioned in Section 4.1, have already been added to the transformation prototype, enabling a more precise evaluation of both user stories and Gherkin scenarios. Below, some of these new patterns are presented:

- (“throw”) *MessageIntermediateEvent* followed by a (“catch”) *TimerIntermediateEvent* followed by an *Activity* in different or same *Lanes*, with the corresponding user story following this format:
US: As “Lane₁” I want to send a(n) “*MessageIntermediateEvent*” to “Pool”, considering that I have to wait for “*TimerIntermediateEvent*” in order to “Lane₂” can “*Activity*”.
- *InclusiveGateway* split connected with “n” *Activities*, with the corresponding user story following this format:
US: As “Lane”, I will to execute “*Activity*_{1..n}” together.
- *ExclusiveGateway* join between an *Activity* and a *MessageEndEvent*, with the corresponding user story following this format:
US: As “Lane” I want to execute “*Activity*” in order to send “*MessageEndEvent*” to outgoing flow depending on “*SequenceFlow*”.

6.1.2. Prototype Coverage

Regarding the transformation prototype, it is important to analyze its capabilities and limitations regarding the extraction of user stories and Gherkin scenarios from BPMN business process models. In this regard, it is relevant to highlight that, in terms of functionalities, the prototype includes the necessary logic to handle business process models containing the following elements, according to BPMN 2.0 notation [5]:

- *Pools*, representing both the process and the external actors communicating with the process itself;
- *Lanes*, representing the internal actors within the process;
- *Tasks*, representing the activities that are part of the process;
- *Events*, representing various event types, including: *StartEvent*, *EndEvent*, *MessageIntermediateEvent* (both Throw and Catch), and *TimerIntermediateEvent* (Catch);
- *Boundary Event*, representing the *ErrorEvent* type;
- *Gateways*, representing Exclusive, Inclusive, and Parallel Gateways.

Therefore, it is evident that the prototype does not yet cover all possible elements, such as other types of gateways, events, among others. Additionally, the approach does not handle subprocesses yet.

6.2. Qualitative Assessment

As previously mentioned, this assessment was conducted using a questionnaire answered by information technology professionals selected based on their consolidated

experience in business process modeling, requirements engineering, and agile methodologies. The participants' expertise enabled them to provide crucial and constructive feedback on the proposed approach. Their evaluations were essential for identifying limitations and opportunities for improvement. The assessment was performed based on the output generated by the transformation tool when applied to a set of BPMN models.

The four BPMN models used in the evaluation were carefully selected to reflect structural diversity and to include a variety of BPMN elements, such as gateways, events, and message flows. Two of these models were sourced from previously published studies, adding realism and practical relevance to the evaluation. Although the number of models is limited, they were chosen to cover a representative set of transformation patterns and to support an initial validation of the approach's feasibility and applicability in practice.

6.2.1. Participants

The questionnaire was answered by five professionals in the field of information technology, each with relevant experience to support a critical evaluation of the proposed approach. Table 2 summarizes their profiles, including identification (ID), years of experience (Exp.) in business process modeling and requirements engineering, familiarity with agile methodologies, and their proficiency in working with user stories.

Table 2. Individual participants' information.

ID	Exp. in Process Modeling	Exp. in Requirements Engineering	Agile Methodologies	Exp. in User Stories
P1	>10 years	>10 years	Kanban, Scrum	Comfortable
P2	6–10 years	>10 years	Scrum	Detailed Analysis
P3	>10 years	>10 years	Scrum	Detailed Analysis
P4	1–5 years	6–10 years	Kanban, Scrum, XP	I Understand Theoretically
P5	1–5 years	1–5 years	Kanban, Scrum	I Understand Theoretically

Two of the five participants reported having over ten years of experience in business process modeling, while two others had between one and five years, and one participant fell within the six to ten year range. Regarding requirements engineering, three participants indicated more than a decade of experience, suggesting a strong foundation for evaluating the proposed solution.

In terms of user story proficiency, two participants stated they were capable of conducting detailed analyses, two reported a theoretical understanding, and one felt comfortable working with this artifact. This distribution indicates that the group possessed a balanced level of expertise aligned with the core focus areas of the study.

All participants had exposure to agile methodologies, particularly Scrum, which is widely adopted in software development and prominently features user stories in requirement specification practices.

Although the number of participants was limited to five, they were carefully selected for their consolidated expertise across the relevant domains. Given the exploratory nature of this study, which focuses on the initial validation of a prototype tool, the evaluation prioritized expert insights rather than statistical generalization. Future research will aim to expand the participant base to enable quantitative analysis and broader empirical substantiation.

6.2.2. Accuracy Analysis

To assess the accuracy of the transformations, participants were asked to evaluate the results of the user story and Gherkin scenario extractions for each of the four BPMN models. Specifically, they answered two questions:

- Question 1—How accurate was the extraction of the user stories, based on the model presented?
- Question 2—How accurate was the extraction of the scenarios, based on the model presented?

The responses revealed a generally positive perception, although with some variability across models. For the first two models, three out of five participants rated the extractions as “*accurate*”, while the remaining two considered them “*not very representative*”. For the third model, four participants deemed the extractions “*accurate*”, and the fourth model received two evaluations of *precision* and two additional “*positive*” evaluations.

In addition to these subjective ratings, the evaluation implicitly incorporated a manual versus automated comparison, since the experts contrasted the tool’s outputs with the artifacts they would normally derive from the same BPMN models. This reflective comparison provided evidence of the tool’s consistency and practical value, even though no formal precision, recall, or coverage metrics were applied at this stage.

Overall, this qualitative design was considered appropriate, as the main evaluation objectives concerned clarity, consistency, and applicability of the generated artifacts, dimensions that are best judged through expert interpretation. Despite the absence of quantitative performance measures, the expert feedback offered valuable insights that guided refinements to both the transformation patterns and the supporting tool.

6.2.3. Feedback Provided by Experts

For each of the models used in this evaluation, as mentioned earlier, four questions were presented to the evaluators, two of which were open-ended (Q3—Question 3 and Q4—Question 4). These are the questions:

- Question 3—Are there any user stories or scenarios you disagree with? If yes, which ones?
- Question 4—What changes would you make to the extracted content (user stories and scenarios)?

Since these are open-ended questions, some evaluators also provided suggestions and changes not specific to the models themselves, but to the approach as a whole, which were incorporated into the final version of both the approach and the tool. We will now discuss these questions for each of the models presented to the evaluators. Upon analyzing the responses of the five evaluators, unanimity was observed on certain points. However, there were divergences in other aspects, prompting us to thoroughly analyze their responses and treat them as suggestions.

Regarding the convergence of the evaluators, four out of five highlighted the definition of possible paths in the exclusive gateway, addressing the issue of natural flow versus alternative paths. At this point, we observed that the solution presented in Section 5.1, which refers to the definition of a specific glossary, was present but lacked a better explanation and greater usability in the proposed tool.

A second issue is directly related to the parallel gateway, where the solution implemented for generating the user story includes all activities until the closing by the Join of the parallel gateway. On this point, we are mapping all possible situations that could prevent the direct and parallel execution of activities from a parallel gateway, as in the following example (see Figure 6), where activities B and C cannot be executed in parallel

due to the dependency on a message from the external world that may not occur at the appropriate time.

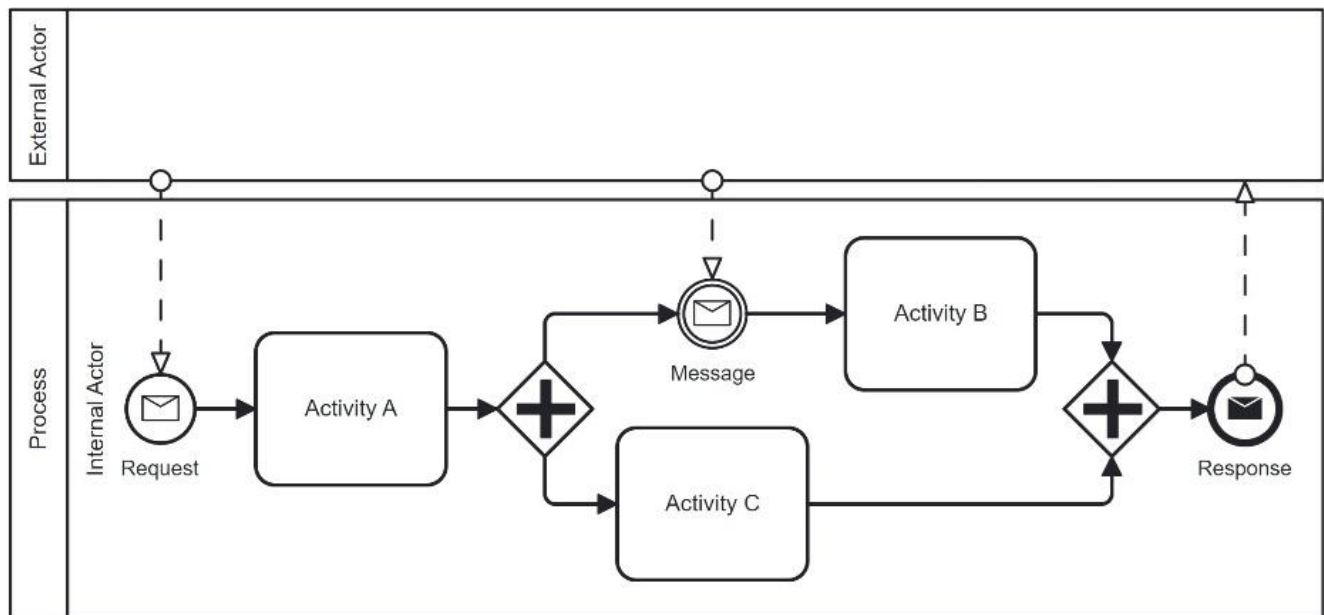


Figure 6. Example of dependency that can prevent the simultaneous execution of two activities.

At various times, from the evaluation of the four models presented to the evaluators, they pointed out the possibility of changing some terms in the user stories templates. To automate this change, we chose to implement the functionality of editing the templates, as shown in Figure 5.

Regarding the Gherkin scenarios, a suggestion that was addressed concerned the “Then” clause, which two evaluators emphasized should include at the end the recipient of the action/notification. Thus, the template “Then “Actor Internal or External” sends/executes a “Notification or Action”” was modified to “Then “Actor Internal or External” sends/executes a “Notification or Action” to the “Actor Internal or External””.

Taking an overall balance, out of the thirteen user stories presented, three contained errors, six had suggested changes, and three new user stories were proposed. Considering that some suggestions from the evaluators were identical or related to the same user story, we concluded that the majority of them are relevant and were considered in this approach to improve the accuracy of user stories extraction and the generated scenarios.

6.2.4. Utility Overview

To assess the usefulness of the presented approach, two new questions were included for the evaluators:

- Question 5—What level of usefulness is attributed to this extraction prototype, in the context of eliciting and specifying requirements?
- Question 6—As a business process designer or requirements engineer, would you use this approach?

In response to the first question, as demonstrated in Figure 7A, two out of five evaluators indicated that the presented transformation tool is highly useful, while other two considered it only interesting. However, one evaluator responded that its usefulness is only regular or normal.

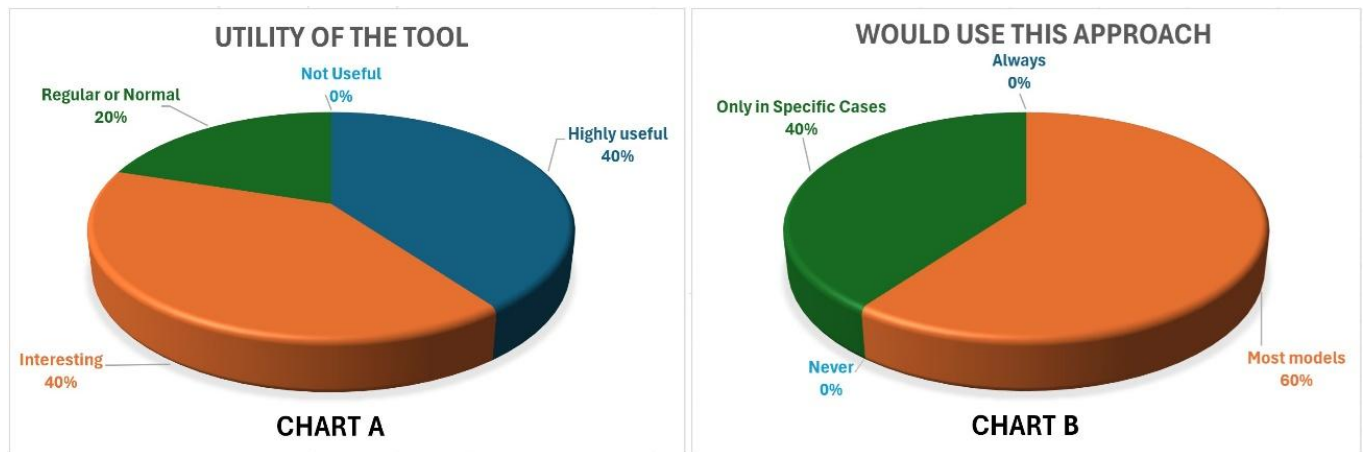


Figure 7. Charts based on the overview of utility.

In Figure 7B, when asked about the usefulness of the approach itself, i.e., the approach used to derive user stories and Gherkin scenarios from business processes, three out of five evaluators indicated that they would use it in most models, while the other two stated they would use it only in specific cases.

Upon analyzing these data, we conclude that the result is positive, providing utility to both the approach and the developed tool. It seems evident that the developed tool has associated potential, although it may not be applicable to all possible business process models and, like all tools, should undergo future improvements. It is important to highlight that its logic is based on the (semi-)automation of the process of extracting information from process models, as evidenced by the responses to the questions, which could be useful and relevant in an organization's processes.

The summary presented in Table 3 reveals a generally positive assessment across all analyzed criteria. The highest average scores were observed in *Relevance to Agile Practices* (4.6) and *Overall Satisfaction* (4.4), indicating the tool's strong alignment with agile methodologies and its perceived practical utility. In contrast, aspects such as *Completeness of Output* and *Customization Possibilities* (both 3.6) suggest room for improvement, particularly regarding broader BPMN coverage and adaptability to diverse business domains. The low standard deviations across responses reinforce the consistency of expert opinions and lend credibility to the evaluation findings.

Table 3. Summary of expert evaluations across the main analysis criteria. Ratings used a 5-point Likert scale (1 = very low, 5 = very high).

Criterion	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5	Mean	Std. Dev.
Perceived Usefulness	4	5	4	4	5	4.4	0.55
Ease of Use	4	4	5	4	4	4.2	0.45
Accuracy of Transformation	5	4	4	4	4	4.2	0.45
Completeness of Output	3	4	4	3	4	3.6	0.55
Relevance to Agile Practices	4	5	5	4	5	4.6	0.55
Customization Possibilities	3	4	4	3	4	3.6	0.55
Overall Satisfaction	4	5	4	4	5	4.4	0.55

6.3. Evaluation Scope and Rationale

The evaluation conducted in this study followed a qualitative, expert-based assessment approach. Five experts in BPMN modeling and requirements engineering were engaged to

provide feedback on the accuracy, completeness, and perceived usefulness of the generated artifacts. This design choice reflects the exploratory stage of the research, in which the primary goal was to validate the conceptual soundness and practical feasibility of the transformation patterns and supporting tool.

We acknowledge that the small sample size limits the statistical generalizability of the results. However, the in-depth, domain-specific feedback collected from experienced practitioners provided valuable insights into the strengths and weaknesses of the approach, guiding refinement of both the transformation logic and the user interface. This type of formative evaluation is considered a suitable first step in the empirical validation of early-stage requirements engineering methods.

In future work, we plan to expand the evaluation to include a larger and more diverse participant base, incorporate objective performance metrics such as precision, recall, and coverage, and conduct comparative studies against manual and semi-automated extraction approaches. We also envision usability testing in real-world project settings to measure time savings, error rates, and stakeholder satisfaction. These steps will enable a more rigorous, quantitative validation to complement the initial qualitative findings presented here.

7. Threats to Validity

After presenting and discussing the evaluation results, it is necessary to assess the potential risks that may affect the validity of the results and their respective conclusions. Next, the threats to validity were discussed according to the guidelines suggested by Wohlin et al. [32]. However, to facilitate understanding, the risks were grouped into three categories: internal validity, external validity, and conclusion validity.

7.1. Internal Validation

Some potential threats to internal validity are related to the questionnaire and the participants. Despite providing contextual information to the evaluators, there is a possibility that the questionnaire could become exhaustive, contain poorly formulated questions, or delve into a specific area in which one of the evaluators might not have much experience or might feel uncomfortable (e.g., Business Process Modeling or Requirements Engineering).

To mitigate these risks, participants were selected based on their prior knowledge in at least one of the relevant areas. Additionally, a pilot test was conducted to validate both the questionnaire and the predefined process models. It is important to note that the results of this pilot test were not used in the final data evaluation; it served solely to make minor adjustments to the models and some questions, thus facilitating the overall understanding of the questionnaire.

7.2. External Validation

One relevant threat to external validity concerns the use of only four specific process models during the evaluation. Although this limited number constrains the generalizability of the findings, the models were carefully selected to ensure structural diversity and to include a variety of BPMN constructs (e.g., tasks, gateways, events, message flows). This diversity allowed the activation and testing of different transformation patterns implemented in the tool. Moreover, two of the four models were extracted from previously published studies, ensuring practical relevance and increasing the external credibility of the evaluation.

Another aspect pertains to the transformation templates used for generating user stories. These templates were initially derived from the process models analyzed during the design phase and, thus, may not comprehensively reflect all industry-specific formats or organizational standards. We acknowledge this limitation and highlight that the trans-

formation tool includes built-in functionality for creating and editing custom templates, allowing organizations to tailor the transformation logic to their specific requirements.

A further consideration involves the language adopted in the tool outputs, which is English. The decision to use English was made to maximize accessibility and facilitate international reuse, given its widespread adoption in global software development contexts. Although this choice may pose challenges for non-native speakers, no language-related difficulties were reported by the participants, who were proficient in English and able to evaluate the generated content effectively.

The limited number of participants in the qualitative assessment also presents a potential threat to external validity. While only five experts participated, each possessed considerable professional experience in business process modeling, requirements engineering, and agile methodologies. Their specialized knowledge enabled meaningful feedback and provided valuable insights for refining the tool and transformation approach. Nonetheless, future evaluations will involve a broader and more heterogeneous sample to enhance representativeness and support statistically grounded conclusions.

Finally, another limitation to external validity is the restricted BPMN element coverage in the current tool implementation. More complex constructs, such as subprocesses, event-based gateways, escalation events, and data objects, are not yet supported. This constraint limits the applicability of the approach in highly complex, real-world process models. However, this design decision was intentional, aiming to first validate the transformation logic using simpler process structures. The tool's modular and extensible architecture was specifically developed to support the gradual incorporation of additional BPMN constructs while ensuring the correctness and scalability of the transformations.

7.3. Conclusion Validation

The limited number of evaluators represents a key constraint on the robustness of the conclusions drawn from this study. A small participant pool increases the risk of bias and reduces the statistical generalizability of the findings. Additionally, the analysis of subjective responses, especially those provided in open-ended questions, can be susceptible to interpretation variability, which may introduce ambiguity or distortion in the results.

To mitigate these concerns in future work, we recommend expanding the evaluator base to include a larger and more diverse sample. This would enhance the representativeness of the feedback, support more robust empirical conclusions, and enable more granular analysis of participant perceptions.

Nonetheless, in the current evaluation, all responses were carefully analyzed, with particular attention given to the interpretation of open-ended qualitative answers. Each evaluator's feedback was reviewed in depth, and specific suggestions for improving the transformation outputs and tool functionality were incorporated, as described in the previous section. This reflective process ensured a meaningful assessment despite the small sample size.

We also recognize that future qualitative evaluations could benefit from adopting structured expert-based assessment methods to deepen the analysis and enhance result reliability. One promising approach is the multi-round Delphi Method, which allows experts to iteratively refine their feedback, reduce individual bias, and converge towards consensus. Incorporating such a method in future evaluations would provide a more robust foundation for interpreting expert judgments. Furthermore, we acknowledge the absence of objective performance metrics, such as precision, recall, or coverage, in the current evaluation. In the present study, the notion of "accuracy" was based on expert judgment and subjective perception rather than on statistically grounded measurements.

Furthermore, we acknowledge the absence of objective performance metrics, such as precision, recall, or coverage, in the current evaluation. In the present study, the notion of “accuracy” was based on expert judgment and subjective perception rather than on statistically grounded measurements. This limitation represents an important gap in the current validation, which constrains the extent to which the results can be generalized.

7.4. Future Directions for Addressing Limitations

While this study focused primarily on frequently used BPMN constructs to ensure a clear and controlled evaluation setting, we acknowledge that the handling of more complex constructs (such as sub-processes, event-based gateways, and boundary events) remains an important direction for future development. The pattern-based architecture and modular tool design already support the incremental incorporation of these constructs, enabling their formalization and testing without disrupting the existing transformation logic.

To strengthen confidence in the scalability of the approach, we plan to design and execute controlled experiments using BPMN repositories that contain complex constructs, such as subprocesses, exception handling, and event-based gateways. These experiments will systematically assess whether the extended transformation patterns maintain correctness, performance, and usability when applied to larger and structurally diverse models. By combining case studies from real-world organizations with stress-testing on synthetic large-scale models, we aim to provide more robust empirical evidence of the approach’s scalability and practical applicability.

Similarly, other limitations identified in this study, such as the coverage of BPMN artifact types, the scalability of the transformation process for very large models, and the limited evaluation scope, are being systematically addressed in our research roadmap. Planned steps include the iterative expansion of the pattern library, performance benchmarking with large-scale BPMN repositories, and broader empirical studies with practitioners from diverse domains.

In addition, future evaluations will explicitly incorporate quantitative performance metrics, such as precision, recall, and coverage, to provide statistically grounded evidence of correctness and completeness. We also intend to establish reference datasets that serve as a ground truth for systematic assessment, enabling direct comparisons between manual and automated derivations of user stories and Gherkin scenarios. Benchmarking against existing approaches will further clarify the relative strengths of our method and enhance its credibility within the broader landscape of automated requirements extraction. These steps will complement the qualitative findings reported in this study and offer a more rigorous empirical foundation for future research and industrial adoption.

Another promising direction for extending the applicability of the approach is the integration with widely used agile management platforms such as Jira and Azure DevOps. Since the generated artifacts can be exported in structured formats (e.g., JSON, CSV), they can be directly imported into these systems via their respective APIs or through intermediate scripts. This capability would enable seamless incorporation of automatically generated user stories and scenarios into existing agile toolchains, fostering adoption in real-world development environments. Future work will explore developing native connectors to streamline this process.

Another relevant direction concerns the progressive formalization of the transformation logic. While the current semi-formal representation was intentionally chosen to maximize usability, future research will explore the specification of transformation patterns using model transformation languages such as ATL or QVT, enabling automated verification and deeper integration with model-driven engineering practices.

These initiatives are intended to extend the scope, robustness, and applicability of the proposed approach, progressively reducing the identified threats to validity and enhancing its value for both research and industrial contexts.

8. Conclusions

This study proposed an automated approach for deriving user stories and Gherkin scenarios from BPMN process models. The solution is grounded in the identification of relationships among the BPMN, user story, and Gherkin metamodels, followed by the specification of transformation patterns to connect these modeling artifacts. A prototype tool was developed to operationalize the approach and execute the transformations automatically. Throughout the development process, iterative testing was conducted to assess both the accuracy and utility of the tool. Subsequently, a qualitative evaluation with domain experts was performed to validate the effectiveness of the proposed solution.

The primary objective was to support the elicitation of functional requirements by narrowing the gap between business process modeling and agile requirements specification. In this regard, the transformation process presented here offers a well-defined mechanism for translating BPMN constructs into artifacts compatible with agile practices, reinforcing the alignment between business and technical domains.

A key contribution of this work is the supporting tool, implemented as a web-based application, which enables the execution, monitoring, and analysis of content extraction tasks. The tool not only operationalizes the proposed transformation patterns but also provides an extensible interface that facilitates the definition of new patterns, allowing for customization across different organizational contexts. Expert feedback collected during the evaluation phase played a critical role in refining the transformation logic and improving the tool's design.

Despite its contributions, the approach presents certain limitations. First, the transformation patterns implemented represent an initial set and may not capture all nuances present in diverse business domains. Although the tool allows users to define additional patterns, expanding this library remains a priority for future work.

Second, the tool currently supports a limited set of BPMN elements. Advanced constructs, such as subprocesses, escalation events, and data objects, were not addressed in the current implementation. This constraint affects the applicability of the tool to more complex or enterprise-level process models. Nevertheless, the tool was architected with extensibility in mind, allowing for the future integration of these elements without compromising consistency or performance.

Another limitation involves the scope of Gherkin scenario extraction. The current approach focuses solely on alternative flows, omitting the generation of scenarios for the main success paths. Although alternative flows add valuable detail to the user stories, the inclusion of main flow scenarios is essential for completeness in agile testing and remains a goal for future iterations.

Additionally, the number of process models and participants included in the evaluation was limited. While expert judgment provided rich insights, broader validation that incorporates a more diverse set of models and practitioners from different roles and industries is necessary to generalize the findings. Future evaluations will also include interviews with project managers and practitioners to strengthen the empirical evidence.

Future work will focus on expanding BPMN coverage to include complex structures, refining transformation logic to incorporate main flow scenarios, and conducting large-scale empirical validations in industrial settings. These efforts aim to enhance the scalability, completeness, and applicability of the proposed approach in real-world software development contexts.

Author Contributions: Conceptualization, D.S.d.S. and J.A.; Methodology, D.S.d.S.; Software, D.M.; Validation, D.M.; Formal analysis, D.M.; Investigation, D.M.; Writing—original draft, D.M.; Writing—review & editing, D.S.d.S. and J.A.; Supervision, D.S.d.S. and J.A.; Project administration, J.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research was generously supported by CNPq (grant number 421085/2023-1) and by FCT/IP (grant number UIDB/04516/2020).

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Dumas, M.; La Rosa, M.; Mendling, J.; Reijers, H.A. *Fundamentals of Business Process Management*, 2nd ed.; Springer: Berlin/Heidelberg, Germany, 2018.
2. Weske, M. *Business Process Management—Concepts, Languages, Architectures*; Springer: Berlin/Heidelberg, Germany, 2012; ISBN 978-3-540-73521-2.
3. Cohn, M. *User Stories Applied: For Agile Software Development*, 1st ed.; Addison-Wesley Professional: Boston, MA, USA, 2004.
4. Gherkin Syntax. Available online: <https://cucumber.io/docs/gherkin/> (accessed on 20 August 2024).
5. OMG-BPMN. Business Process Model and Notation (2.0.2). Technical Report, Object Management Group. 2014. Available online: <https://www.omg.org/spec/BPMN> (accessed on 4 August 2024).
6. Leopold, H.; Mendling, J. Polyvyanyy, Supporting process model validation through natural language generation. *IEEE Trans. Softw. Eng.* **2014**, *40*, 818–840. [CrossRef]
7. Aysolmaz, B.; Leopold, H.; Reijers, H.A.; Demirörs, O. A semi-automated approach for generating natural language requirements documents based on business process models, Information and Software Technology. *Inf. Softw. Technol.* **2018**, *93*, 14–29. [CrossRef]
8. Palmer, N. *XML Process Definition Language*; Springer: New York, NY, USA, 2009; p. 3601.
9. OMG-UML. Unified Modeling Language (2.5.1). Technical Report, Object Management Group. 2017. Available online: <https://www.omg.org/spec/UML/About-UML/> (accessed on 4 August 2024).
10. Sommerville, I. *Software Engineering*, 10th ed.; Pearson: London, UK, 2021.
11. Abrahamsson, P.; Salo, O.; Ronkainen, J.; Warsta, J. Agile software development methods: Review and analysis. *arXiv* **2017**, arXiv:1709.08439. [CrossRef]
12. Pokharel, P.; Vaidya, P. A study of user story in practice. In Proceedings of the 2020 International Conference on Data Analytics for Business and Industry: Way Towards a Sustainable Economy (ICDABI), Sakheer, Bahrain, 26–27 October 2020; pp. 1–5.
13. Necula, S.-C.; Dumitriu, F.; Greavu-Şerban, V. A Systematic Literature Review on Using Natural Language Processing in Software Requirements Engineering. *Electronics* **2024**, *13*, 2055. [CrossRef]
14. Delgado, A.; Calejari, D.; García, F.; Weber, B. Model-driven management of BPMN-based business process families. *Softw. Syst. Model* **2022**, *21*, 2517–2553. [CrossRef]
15. Cardoso, E.; Almeida, J.P.; Guizzardi, G. Requirements engineering based on business process models: A case study. In Proceedings of the 2009 13th Enterprise Distributed Object Computing Conference Workshops, Auckland, New Zealand, 1–4 September 2009; pp. 320–327.
16. Ma, Q.; Jiang, Y. Process-oriented information system requirements engineering—A case study. *J. Bus. Cases Appl.* **2014**, *10*, 1–16.
17. Mayr, H.C.; Kop, C.; Esberger, D. Business Process Modeling and Requirements Modeling. In Proceedings of the First International Conference on the Digital Society (ICDS'07), Guadeloupe, France, 2–6 January 2007; p. 8.
18. Li, J.; Jeffery, R.; Fung, K.H.; Zhu, L.; Wang, Q.; Zhang, H.; Xu, X. A business process-driven approach for requirements dependency analysis. In *Business Process Management*; Springer: Berlin/Heidelberg, Germany, 2012; Volume 7481, pp. 200–215.
19. Demirörs, O.; Gencel, Ç.; Tarhan, A. Utilizing business process models for requirements elicitation. In Proceedings of the 2003 Proceedings 29th Euromicro Conference, Belek-Antalya, Turkey, 1–6 September 2003; pp. 1–4.
20. Monsalve, C.; April, A.; Abran, A. Requirements Elicitation Using BPM Notations: Focusing on the Strategic Level Representation. In Proceedings of the 10th WSEAS international conference on Applied computer and applied computational science, Venice, Italy, 8–10 March 2011; pp. 235–241.
21. González, J.D.L.V.; Díaz, J. Business process-driven requirements engineering: A goal-based approach. In Proceedings of the VIII International Workshop on Business Process Modeling, Development and Support (BPMDS'07), Trondheim, Norway, 11–12 June 2007; pp. 1–9.

22. Cox, K.; Phalp, K.T.; Bleistein, S.J.; Verner, J.M. Deriving requirements from process models via the problem frames approach. *Inf. Softw. Technol.* **2005**, *47*, 319–337. [[CrossRef](#)]
23. Malik, S.; Bajwa, I.S. Back to origin: Transformation of business process models to business rules. In *Business Process Management Workshops*; La Rosa, M., Soffer, P., Eds.; Springer: Berlin/Heidelberg, Germany, 2013; pp. 611–622.
24. Türetken, O.; Su, O.; Demirs, O. Automating software requirements generation from business process models. In *Proceedings of the Principles of Software Engineering (PRISE'04)*, Buenos Aires, Argentina, 25–27 September 2004; pp. 1–16.
25. Coşkunçay, A.; Aysolmaz, B.; Demirs, O.; Bilen, O.; Dogan, I. Bridging the gap between business process modeling and software requirements analysis: A case study. In *Proceedings of the 5th Mediterranean Conference on Information Systems (MCIS)*, Tel Aviv, Israel, 12–14 September 2010; p. 20.
26. Herden, A.; Farias, P.P.M.; Albuquerque, A. An approach based on BPMN to detail use cases. In *New Trends in Networking, Computing, Elearning, Systems Sciences, and Engineering*; Springer: Berlin/Heidelberg, Germany, 2015; pp. 537–544.
27. Gordijn, J.; Akkermans, J.M. Designing and evaluating e-business models. *IEEE Intell. Syst.* **2001**, *16*, 11–17. [[CrossRef](#)]
28. Yu, E. Towards Modelling and Reasoning Support for Early-Phase Requirements Engineering. In *Proceedings of the 3rd IEEE International Symposium on Requirements Engineering*, Annapolis, MD, USA, 6–10 January 1997; pp. 226–235. [[CrossRef](#)]
29. Kourani, H.; Berti, A.; Schuster, D.; van der Aalst, W.M.P. Process Modeling with Large Language Models. In *Enterprise, Business-Process and Information Systems Modeling. BPMDS EMMSAD 2024*; van der Aa, H., Bork, D., Schmidt, R., Sturm, A., Eds.; Lecture Notes in Business Information Processing; Springer: Cham, Switzerland, 2024; Volume 511. [[CrossRef](#)]
30. Dumas, M.; Fournier, F.; Limonad, L.; Marrella, A.; Montali, M.; Rehse, J.R.; Accorsi, R.; Calvanese, D.; De Giacomo, G.; Fahland, D.; et al. AI-augmented Business Process Management Systems: A Research Manifesto. *ACM Trans. Manag. Inf. Syst.* **2023**, *14*, 1–19. [[CrossRef](#)]
31. Camunda. bpmn-js Walkthrough. Acedido a Set 11, 2023. Available online: <https://bpmn.io/toolkit/bpmn-js/walkthrough/> (accessed on 4 August 2024).
32. Wohlin, C.; Runeson, P.; Höst, M.; Ohlsson, M.C.; Regnell, B.; Wesslén, A. *Experimentation in Software Engineering*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2012.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.